

Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms

MICHAEL L. FREDMAN

University of California, San Diego, La Jolla, California

AND

ROBERT ENDRE TARJAN

AT&T Bell Laboratories, Murray Hill, New Jersey

Abstract. In this paper we develop a new data structure for implementing heaps (priority queues). Our structure, *Fibonacci heaps* (abbreviated *F-heaps*), extends the binomial queues proposed by Vuillemin and studied further by Brown. F-heaps support arbitrary deletion from an n -item heap in $O(\log n)$ amortized time and all other standard heap operations in $O(1)$ amortized time. Using F-heaps we are able to obtain improved running times for several network optimization algorithms. In particular, we obtain the following worst-case bounds, where n is the number of vertices and m the number of edges in the problem graph:

- (1) $O(n \log n + m)$ for the single-source shortest path problem with nonnegative edge lengths, improved from $O(m \log_{(m/n+2)} n)$;
- (2) $O(n^2 \log n + nm)$ for the all-pairs shortest path problem, improved from $O(nm \log_{(m/n+2)} n)$;
- (3) $O(n^2 \log n + nm)$ for the assignment problem (weighted bipartite matching), improved from $O(nm \log_{(m/n+2)} n)$;
- (4) $O(m\beta(m, n))$ for the minimum spanning tree problem, improved from $O(m \log \log_{(m/n+2)} n)$, where $\beta(m, n) = \min \{i \mid \log^i n \leq m/n\}$. Note that $\beta(m, n) \leq \log^* n$ if $m \geq n$.

Of these results, the improved bound for minimum spanning trees is the most striking, although all the results give asymptotic improvements for graphs of appropriate densities.

Categories and Subject Descriptors: E.1 [Data]: Data Structures—*trees; graphs*; F.2.2 [Analysis of Algorithms and Problem Complexity]: Nonnumerical Algorithms and Problems—*computations on discrete structures; sorting and searching*; G.2.2 [Discrete Mathematics]: Graph Theory—*graph algorithms; network problems; trees*

General Terms: Algorithms, Theory

Additional Key Words and Phrases: Heap, matching, minimum spanning tree, priority queue, shortest path

A preliminary version of this paper appeared in the *Proceedings of the 25th IEEE Symposium on the Foundations of Computer Science* (Singer Island, Fla., Oct. 24–26). IEEE, New York, 1984, pp. 338–346, © IEEE. Any portion of this paper that appeared in the preliminary version is reprinted with permission.

This research was supported in part by the National Science Foundation under grant MCS 82-04031.

Authors' addresses: M. L. Fredman, Electrical Engineering and Computer Science Department, University of California, San Diego, La Jolla, CA 92093; R. E. Tarjan, AT&T Bell Laboratories, Murray Hill, NJ 07974.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1987 ACM 0004-5411/87/0700-0596 \$1.50

1. Introduction

A *heap* is an abstract data structure consisting of a set of *items*, each with a real-valued *key*, subject to the following operations:

<i>make heap</i> :	Return a new, empty heap.
<i>insert</i> (i, h):	Insert a new item i with predefined key into heap h .
<i>find min</i> (h):	Return an item of minimum key in heap h . This operation does not change h .
<i>delete min</i> (h):	Delete an item of minimum key from heap h and return it.

In addition, the following operations on heaps are often useful:

<i>meld</i> (h_1, h_2):	Return the heap formed by taking the union of the item-disjoint heaps h_1 and h_2 . This operation destroys h_1 and h_2 .
<i>decrease key</i> (Δ, i, h):	Decrease the key of item i in heap h by subtracting the nonnegative real number Δ . This operation assumes that the position of i in h is known.
<i>delete</i> (i, h):	Delete arbitrary item i from heap h . This operation assumes that the position of i in h is known.

Note that other authors have used different terminology for heaps. Knuth [15] called heaps “priority queues.” Aho et al. [1] used this term for heaps not subject to melding and called heaps subject to melding “mergeable heaps.”

In our discussion of heaps, we assume that a given item is in only one heap at a time and that a pointer to its heap position is maintained. It is important to remember that heaps do *not* support efficient searching for an item.

Remark. The heap parameter h in *decrease key* and *delete* is actually redundant, since item i determines h . However, our implementations of these operations require direct access to h , which must be provided by an auxiliary data structure if h is not a parameter to the operation. If melding is allowed, a data structure for disjoint set union [26] must be used for this purpose; otherwise, a pointer from each item to the heap containing it suffices.

Vuillemin [27] invented a class of heaps, called *binomial queues*, that support all the heap operations in $O(\log n)$ worst-case time. Here n is the number of items in the heap or heaps involved in the operation. Brown [2] studied alternative representations of binomial heaps and developed both theoretical and experimental running-time bounds. His results suggest that binomial queues are a good choice in practice if meldable heaps are needed, although several other heap implementations have the same $O(\log n)$ worst-case time bound. For further discussion of heaps, see [1], [2], [15], and [24].

In this paper we develop an extension of binomial queues called *Fibonacci heaps*, abbreviated *F-heaps*. F-heaps support *delete min* and *delete* in $O(\log n)$ amortized time, and all the other heap operations, in particular *decrease key*, in $O(1)$ amortized time. For situations in which the number of deletions is small compared to the total number of operations, F-heaps are asymptotically faster than binomial queues.

Heaps have a variety of applications in network optimization, and in many such applications, the number of deletions is relatively small. Thus we are able to use F-heaps to obtain asymptotically faster algorithms for several well-known network optimization problems. Our original purpose in developing F-heaps was to speed

up Dijkstra's algorithm for the single-source shortest path problem with non-negative length edges [5]. Our implementation of Dijkstra's algorithm runs in $O(n \log n + m)$ time, improved from Johnson's $O(m \log_{(m/n+2)} n)$ bound [13, 24].

Various other network algorithms use Dijkstra's algorithm as a subroutine, and for each of these, we obtain a corresponding improvement. Thus we can solve both the all-pairs shortest path problem with possibly negative length edges and the assignment problem (bipartite weighted matching) in $O(n^2 \log n + nm)$ time, improved from $O(nm \log_{(m/n+2)} n)$ [24].

We also obtain a faster method for computing minimum spanning trees. Our bound is $O(m\beta(m, n))$, improved from $O(m \log \log_{(m/n+2)} n)$ [4, 24], where

$$\beta(m, n) = \min\{i \mid \log^{(i)} n \leq m/n\}.$$

All our bounds for network optimization are asymptotic improvements for graphs of intermediate density ($n \ll m \ll n^2$). Our bound for minimum spanning trees, which is perhaps our most striking result, is an asymptotic improvement for sparse graphs as well.

The remainder of the paper consists of five sections. In Section 2 we develop and analyze F-heaps. In Section 3 we discuss variants of F-heaps and some additional heap operations. In Section 4 we use F-heaps to implement Dijkstra's algorithm. In Section 5 we discuss the minimum spanning tree problem. In Section 6 we mention several more recent results and remaining open problems.

2. Fibonacci Heaps

To implement heaps we use heap-ordered trees. A *heap-ordered tree* is a rooted tree containing a set of items, one item in each node, with the items arranged in *heap order*: If x is any node, then the key of the item in x is no less than the key of the item in its parent $p(x)$, provided x has a parent. Thus the tree root contains an item of minimum key. The fundamental operation on heap-ordered trees is *linking*, which combines two item-disjoint trees into one. Given two trees with roots x and y , we link them by comparing the keys of the items in x and y . If the item in x has the smaller key, we make y a child of x ; otherwise, we make x a child of y . (See Figure 1.)

A *Fibonacci heap* (*F-heap*) is a collection of item-disjoint heap-ordered trees. We impose no explicit constraints on the number or structure of the trees; the only constraints are implicit in the way the trees are manipulated. We call the number of children of a node x its *rank* $r(x)$. There is no constant upper bound on the rank of a node, although we shall see that the rank of a node with n descendants is $O(\log n)$. Each node is either *marked* or *unmarked*; we shall discuss the use of marking later.

In order to make the correctness of our claimed time bounds obvious, we assume the following representation of F-heaps: Each node contains a pointer to its parent (or to a special node *null* if it has no parent) and a pointer to one of its children. The children of each node are doubly linked in a circular list. Each node also contains its rank and a bit indicating whether it is marked. The roots of all the trees in the heap are doubly linked in a circular list. We access the heap by a pointer to a root containing an item of minimum key; we call this root the *minimum node* of the heap. A minimum node of null denotes an empty heap. (See Figure 2.) This representation requires space in each node for one item, four pointers, an integer, and a bit. The double linking of the lists of roots and children makes deletion from such a list possible in $O(1)$ time. The circular linking makes concatenation of lists possible in $O(1)$ time.

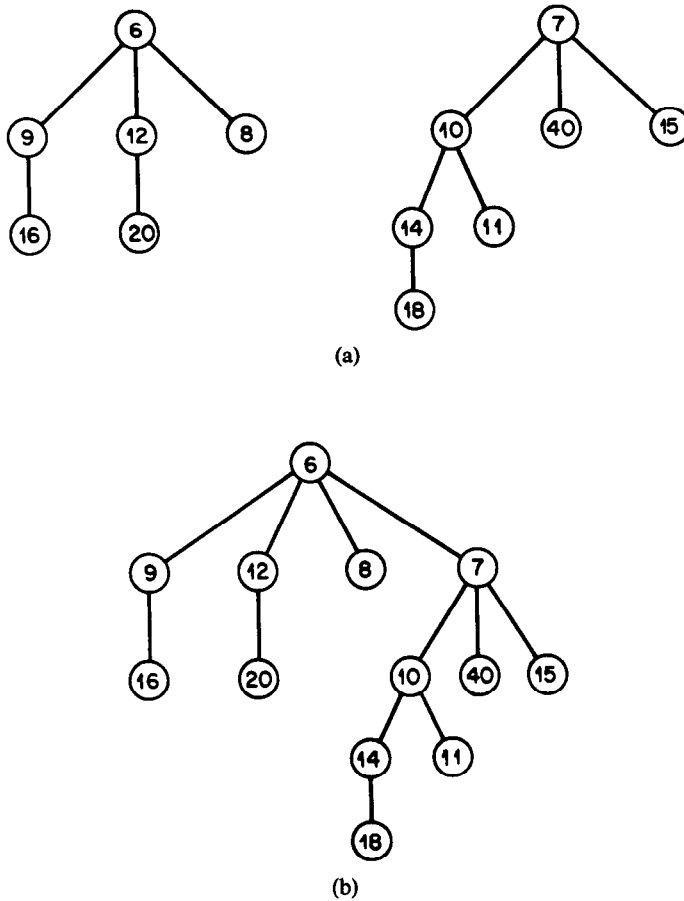


FIG. 1. Linking two heap-ordered trees. (In this and most of the succeeding figures, we do not distinguish between items and their keys.) (a) Two trees. (b) After linking.

We shall postpone a discussion of *decrease key* and *delete* until later in the section. The remaining heap operations we carry out as follows: To perform *make heap*, we return a pointer to null. To perform *find min* (h), we return the item in the minimum node of h . To carry out *insert* (i, h), we create a new heap consisting of one node containing i , and replace h by the meld of h and the new heap. To carry out *meld* (h_1, h_2), we combine the root lists of h_1 and h_2 into a single list, and return as the minimum node of the new heap either the minimum node of h_1 , or the minimum node of h_2 , whichever contains the item of the smaller key. (In the case of a tie, the choice is arbitrary.) All of these operations take $O(1)$ time.

The most time-consuming operation is *delete min* (h). We begin the deletion by removing the minimum node, say, x , from h . Then we concatenate the list of children of x with the list of roots of h other than x , and repeat the following step until it no longer applies.

Linking Step. Find any two trees whose roots have the same rank, and link them. (The new tree root has rank one greater than the ranks of the old tree roots.)

Once there are no two trees with roots of the same rank, we form a list of the remaining roots, in the process finding a root containing an item of minimum key

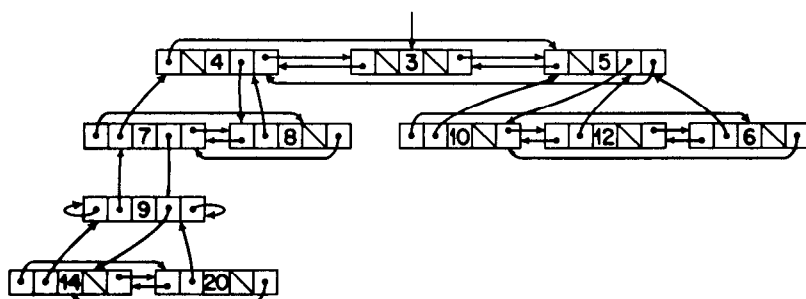


FIG. 2. Pointers representing an F-heap. The four pointers in each node indicate the left sibling, the parent, some child, and the right sibling. The middle field in each node is its key. Ranks and mark bits are not shown.

to serve as the minimum node of the modified heap. We complete the deletion by saving the item in x , destroying x , and returning the saved item. (See Figure 3.)

The *delete min* operation requires finding pairs of tree roots of the same rank to link. To do this we use an array indexed by rank, from zero up to the maximum possible rank. Each array position holds a pointer to a tree root. When performing a *delete min* operation, after deleting the minimum node and forming a list of the new tree roots, we insert the roots one by one into the appropriate array positions. Whenever we attempt to insert a root into an already occupied position, we perform a linking step and attempt to insert the root of the new tree into the next higher position. After successfully inserting all the roots, we scan the array, emptying it. The total time for the *delete min* operation is proportional to the maximum rank of any of the nodes manipulated plus the number of linking steps.

The data structure we have so far described is a “lazy melding” version of binomial queues. If we begin with no heaps and carry out an arbitrary sequence of heap operations (not including *delete* or *decrease key*), then each tree ever created is a *binomial tree*, defined inductively as follows: A binomial tree of rank zero consists of a single node; a binomial tree of rank $k > 0$ is formed by linking two binomial trees of rank $k - 1$. (See Figure 4.) A binomial tree of rank k contains exactly 2^k nodes, and its root has exactly k children. Thus every node in an n -item heap has rank at most $\log n$.¹

We can analyze the amortized running times of the heap operations by using the “potential” technique of Sleator and Tarjan [19, 25]. We assign to each possible collection of heaps a real number called the *potential* of the heaps. We define the *amortized time* of a heap operation to be its actual running time plus the net increase it causes in the potential. (A decrease in potential counts negatively and thus makes the amortized time less than the actual time.) With this definition, the actual time of a sequence of operations is equal to the total amortized time plus the net decrease in potential over the entire sequence.

To apply this technique, we define the potential of a collection of heaps to be the total number of trees they contain. If we begin with no heaps, the initial potential is zero, and the potential is always nonnegative. Thus the total amortized time of a sequence of operations is an upper bound on the total actual time. The amortized time of a *make heap*, *find min*, *insert*, or *meld* operation is $O(1)$: An insertion increases the number of trees by one; the other operations do not affect

¹ All logarithms in this paper for which a base is not explicitly specified are base 2.

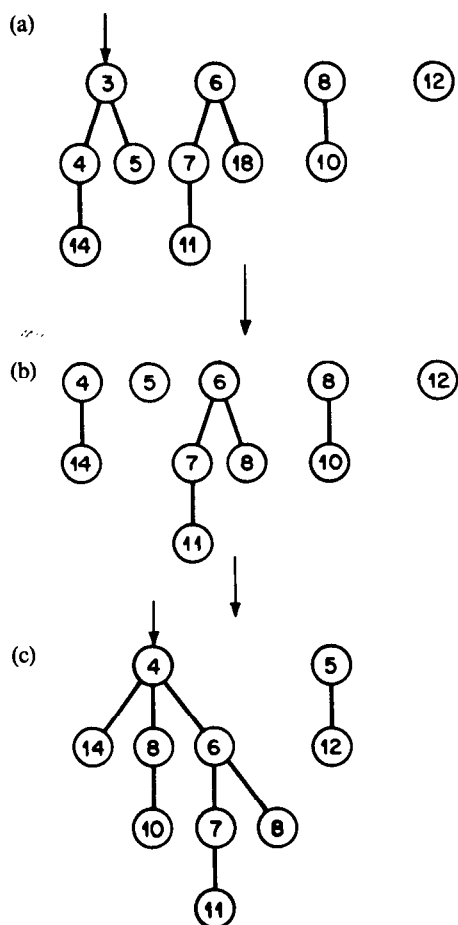


FIG. 3. Effect of a *delete min* operation. (a) Before the deletion. (b) After removal of the minimum node, containing 3. (c) After linking 4 and 8, then 4 and 6, then 5 and 12.

the number of trees. If we charge one unit of time for each linking step, then a *delete min* operation has an amortized time of $O(\log n)$, where n is the number of items in the heap: Deleting the minimum node increases the number of trees by at most $\log n$; each linking step decreases the number of trees by one.

Our goal now is to extend this data structure and its analysis to include the remaining heap operations. We implement *decrease key* and *delete* as follows: To carry out *decrease key* (Δ, i, h), we subtract Δ from the key of i , find the node x containing i , and cut the edge joining x to its parent $p(x)$. This requires removing x from the list of children of $p(x)$ and making the parent pointer of x null. The effect of the cut is to make the subtree rooted at x into a new tree of h , and requires decreasing the rank of $p(x)$ and adding x to the list of roots of h . (See Figure 5.) (If x is originally a root, we carry out *decrease key* (Δ, i, h) merely by subtracting Δ from the key of i .) If the new key of i is smaller than the key of the minimum node, we redefine the minimum node to be x . This method works because Δ is nonnegative; decreasing the key of i preserves heap order within the subtree rooted at x , though it may violate heap order between x and its parent. A *decrease key* operation takes $O(1)$ actual time.

The *delete* operation is similar to *decrease key*. To carry out *delete* (i, h), we find the node x containing i , cut the edge joining x and its parent, form a new list of roots by concatenating the list of children of x with the original list of roots, and

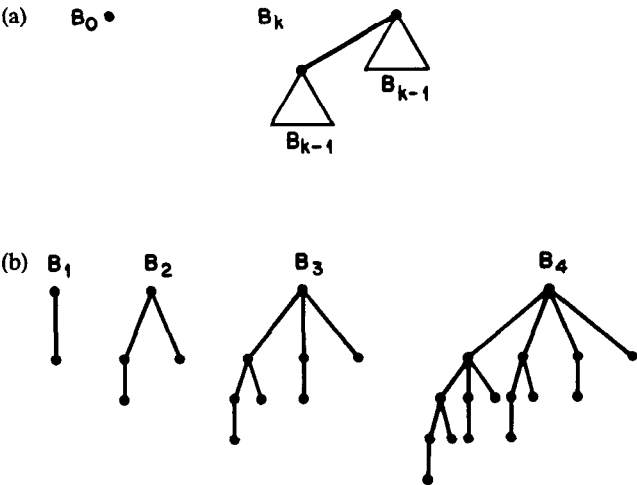


FIG. 4. Binomial trees. (a) Inductive definition. (b) Examples.

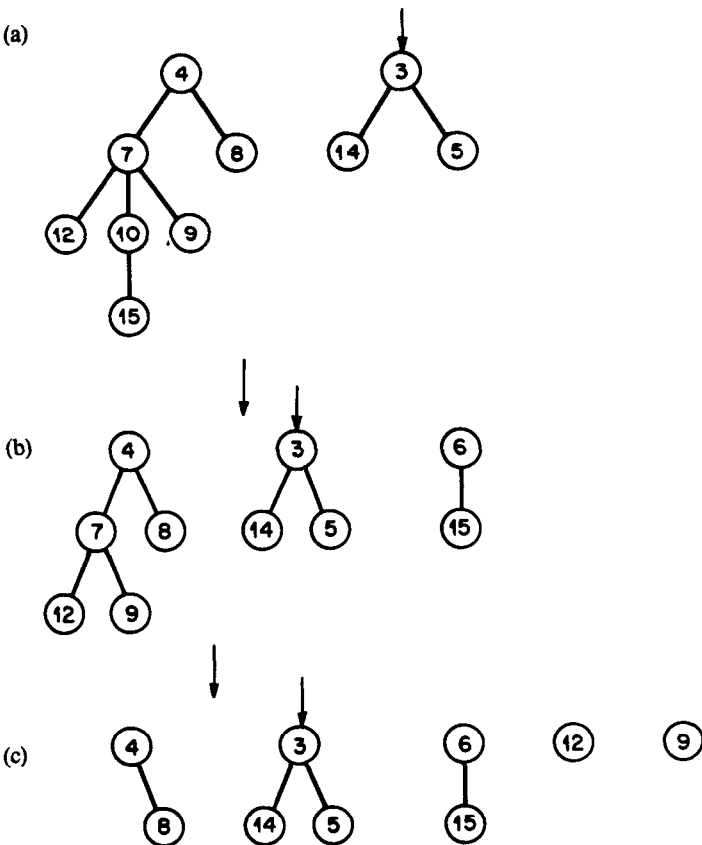


FIG. 5. The *decrease key* and *delete* operations. (a) The original heap. (b) After reducing key 10 to 6. The minimum node is still the node containing 3. (c) After deleting key 7.

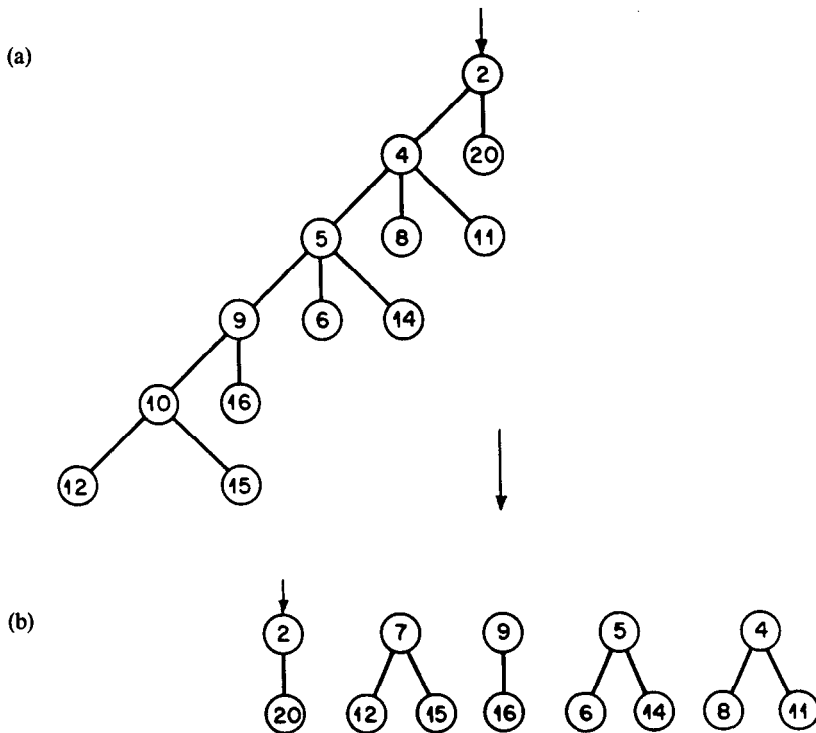


FIG. 6. Cascading cuts. (a) Just before decreasing key 10. Nodes 4, 5, and 9 are assumed to have previously lost a child via a cut. (b) After decreasing key 10 to 7. The original cut separates 10 from 9. Cascading cuts separate 9 from 5, 5 from 4, and 4 from 2.

destroy node x . (See Figure 5.) (If x is originally a root, we remove it from the list of roots rather than removing it from the list of children of its parent; if x is the minimum node of the heap, we proceed as in *delete min*.) A *delete* operation takes $O(1)$ actual time, unless the node destroyed is the minimum node.

There is one additional detail of the implementation that is necessary to obtain the desired time bounds. After a root node x has been made a child of another node by a linking step, as soon as x loses two of its children through cuts, we cut the edge joining x and its parent as well, making x a new root as in *decrease key*. We call such a cut a *cascading cut*. A single *decrease key* or *delete* operation in the middle of a sequence of operations can cause a possibly large number of cascading cuts. (See Figure 6.)

The purpose of marking nodes is to keep track of where to make cascading cuts. When making a root node x a child of another node in a linking step, we unmark x . When cutting the edge joining a node x and its parent $p(x)$, we decrease the rank of $p(x)$ and check whether $p(x)$ is a root. If $p(x)$ is not a root, we mark it if it is unmarked and cut the edge to its parent if it is marked. (The latter case may lead to further cascading cuts.) With this method, each cut takes $O(1)$ time.

This completes the description of F-heaps. Our analysis of F-heaps hinges on two crucial properties: (1) Each tree in an F-heap, even though not necessarily a binomial tree, has a size at least exponential in the rank of its root; and (2) the number of cascading cuts that take place during a sequence of heap operations is bounded by the number of *decrease key* and *delete* operations. Before proving

these properties, we remark that cascading cuts are introduced in the manipulation of F-heaps for the purpose of preserving property (1). Moreover, the condition for their occurrence, namely, the “loss of two children” rule, limits the frequency of cascading cuts as described by property (2). The following lemma implies property (1):

LEMMA 1. *Let x be any node in an F-heap. Arrange the children of x in the order they were linked to x , from earliest to latest. Then the i th child of x has a rank of at least $i - 2$.*

PROOF. Let y be the i th child of x , and consider the time when y was linked to x . Just before the linking, x had at least $i - 1$ children (some of which it may have lost after the linking). Since x and y had the same rank just before the linking, they both had a rank of at least $i - 1$ at this time. After the linking, the rank of y could have decreased by at most one without causing y to be cut as a child of x . $\square$

COROLLARY 1. *A node of rank k in an F-heap has at least $F_{k+2} \geq \phi^k$ descendants, including itself, where F_k is the k th Fibonacci number ($F_0 = 0$, $F_1 = 1$, $F_k = F_{k-2} + F_{k-1}$ for $k \geq 2$), and $\phi = (1 + \sqrt{5})/2$ is the golden ratio. (See Figure 7.)*

PROOF. Let S_k be the minimum possible number of descendants of a node of rank k . Obviously, $S_0 = 1$, and $S_1 = 2$. Lemma 1 implies that $S_k \geq \sum_{i=0}^{k-2} S_i + 2$ for $k \geq 2$. The Fibonacci numbers satisfy $F_{k+2} = \sum_{i=2}^k F_i + 2$ for $k \geq 2$, from which $S_k \geq F_{k+2}$ for $k \geq 0$ follows by induction on k . The inequality $F_{k+2} \geq \phi^k$ is well known [14]. $\square$

Remark. This corollary is the source of the name “Fibonacci heap.”

To analyze F-heaps we need to extend our definition of potential. We define the potential of a collection of F-heaps to be the number of trees plus twice the number of marked nonroot nodes. The $O(1)$ amortized time bounds for *make heap*, *find min*, *insert*, and *meld* remain valid, as does the $O(\log n)$ bound for *delete min*; *delete min (h)* increases the potential by at most $1.4404 \log n$ minus the number of linking steps, since, if the minimum node has rank k , then $\phi^k \leq n$ and thus $k \leq \log n / \log \phi \leq 1.4404 \log n$.

Let us charge one unit of time for each cut. A *decrease key* operation causes the potential to increase by at most three minus the number of cascading cuts, since the first cut converts a possibly unmarked nonroot node into a root, each cascading cut converts a marked nonroot node into a root, and the last cut (either first or cascading) can convert a nonroot node from unmarked to marked. It follows that *decrease key* has an $O(1)$ amortized time bound. Combining the analysis of *decrease key* with that of *delete min*, we obtain an $O(\log n)$ amortized time bound for *delete*. Thus we have the following theorem:

THEOREM 1. *If we begin with no F-heaps and perform an arbitrary sequence of F-heap operations, then the total time is at most the total amortized time, where the amortized time is $O(\log n)$ for each *delete min* or *delete* operation and $O(1)$ for each of the other operations.*

We close this section with a few remarks on the storage utilization of F-heaps. Our implementation of F-heaps uses four pointers per node, but this can be reduced to three per node by using an appropriate alternative representation [2] and even to two per node by using a more complicated representation, at a cost of a constant factor in running time. Although our implementation uses an array for finding roots of the same rank to link, random-access memory is not actually necessary

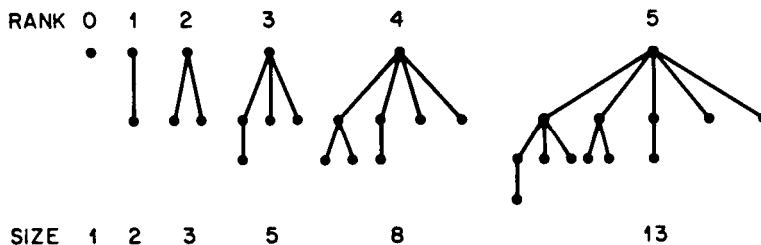


FIG. 7. Trees of minimum possible size for a given rank in a Fibonacci heap.

for this purpose. Instead, we can maintain a doubly linked list of *rank nodes* representing the possible ranks. Each node has a *rank pointer* to the rank node representing its rank. Since the rank of a node is initially zero and only increases or decreases by one, it is easy to maintain the rank pointers. When we need to carry out linking steps, we can use each rank node to hold a pointer to a root of the appropriate rank. Thus the entire data structure can be implemented on a pointer machine [22] with no loss in asymptotic efficiency.

3. Variants of Fibonacci Heaps

In this section we consider additional heap operations and four variants of F-heaps designed to accommodate them. We begin with a closer look at deletion of arbitrary items. The $O(\log n)$ time bound for deletion derived in Section 2 can be an overestimate in some situations. For example, we can delete all the items in an n -item heap in $O(n)$ time, merely by starting from the minimum node and traversing all the trees representing the heap, dismantling them as we go. This observation generalizes to a mechanism for “lazy” deletion, due to Cheriton and Tarjan [4]. This idea applied to F-heaps gives our first variant, *F-heaps with vacant nodes*, which we shall now describe.

We perform a *delete min* or *delete* operation merely by removing the item to be deleted from the node containing it, leaving a vacant node in the data structure (which if necessary we can mark vacant). Now deletions take only $O(1)$ time, but we must modify the implementations of *meld* and *find min* since in general the minimum node in a heap may be vacant. When performing *meld*, if one of the heaps to be melded has a vacant minimum node, this node becomes the minimum node of the new heap. To perform *find min* (h) if the minimum node is vacant, we traverse the trees representing the heap top-down, destroying all vacant nodes reached during the traversal and not continuing the traversal below any nonvacant node. This produces a set of trees all of whose roots are nonvacant, which we then link as in the original implementation of *delete min*. (See Figure 8.) The following lemma bounds the amortized time of *find min*.

LEMMA 2. *A find min operation takes $O(l(\log(n/l) + 1))$ amortized time, where l is the number of vacant nodes destroyed and n is the number of nodes in the heap; if $l = 0$, the amortized time is $O(1)$.*

PROOF. If $l = 0$, the lemma is obvious. Thus suppose $l \geq 1$. The amortized time of the *find min* operation is at most a constant times $\log n$ plus the number of new trees created by the destruction of vacant nodes. Let x be any destroyed vacant node, and suppose that x has k nonvacant children before its destruction. By Lemma 1, at least one of these, say, y , has a rank of at least $k - 2$, which means

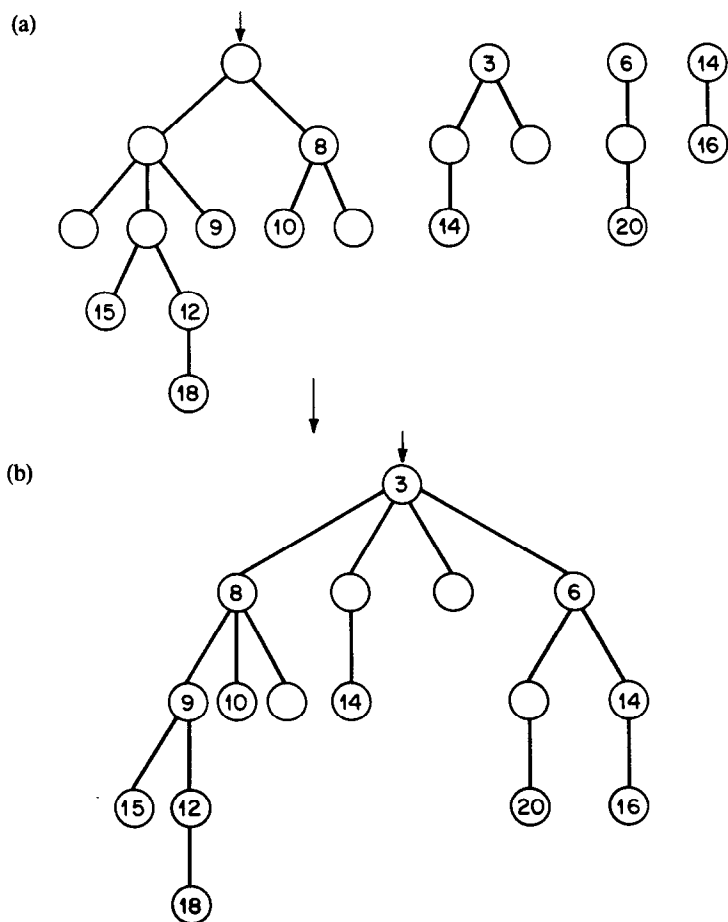


FIG. 8. A *find min* operation on a heap with vacant nodes. (a) The original heap. (b) After *find min*. Subtrees with roots 15, 12, 9, and 8 become trees, and then linking takes place.

that y is the root of a subtree containing at least ϕ^{k-2} nodes. These ϕ^{k-2} nodes can be uniquely associated with x . In other words, if the l destroyed vacant nodes have $k_1, k_2, \dots, k_l$ nonvacant children, then $\sum_{i=1}^l \phi^{k_i-2} \leq n$. Subject to this constraint, the sum of the k_i s, which counts the number of new trees created, is maximized when all the k_i s are equal. This implies that $\sum_{i=1}^l k_i = O(l(\log(n/l) + 1))$, giving the lemma. $\square$

Lazy deletion is especially useful for applications in which the deleted items can be identified implicitly, as, for example, if there is a predicate that specifies whether an item is deleted. The main drawback with lazy deletion is that it may use extra space if individual items are inserted and deleted many times. We can avoid this drawback by changing the data structure slightly, giving our second variant of F-heaps, called *F-heaps with good and bad trees*.

In the new variant, we avoid the use of vacant nodes. Instead, we divide the trees comprising the F-heap into two groups: *good trees* and *bad trees*. We maintain the minimum node to be a root of minimum key among the good trees. When inserting an item, the corresponding one-node tree becomes a good tree in the heap. When

melding two heaps, we combine their sets of good trees and their sets of bad trees. When performing a *decrease key* operation, all the new trees formed by cascading cuts become good trees. We carry out a *delete* operation as described in Section 2 except that, if the deleted item is the minimum node of the heap, all the subtrees rooted at its children become bad trees; we delay any linking until the next *find min*. (This includes the case of a *delete min* operation.) To carry out a *find min*, we check whether there are any bad trees. If not, we merely return the minimum node. If so, we link trees whose roots have equal rank until there are no two trees of equal rank, make all trees good, update the minimum node, and return it.

With this variant of F-heaps, we obtain the following amortized time bounds (using an analysis similar to that above): $O(1)$ for *find min*, *insert*, *meld*, and *decrease key*; and $O(\log(n/l) + 1)$ per *delete* or *delete min* operation for a sequence of l such operations not separated by a *find min*. The advantage of this variant is that it requires no space for vacant nodes. The disadvantage is that it does not support implicit deletion.

Our third variant of F-heaps uses only a single tree to represent each heap. In this *one-tree variant*, we mark each node that is not a root as either *good* or *bad*; the node type is determined when the linking operation that makes the node a nonroot is done. These marks are in addition to the marks used for cascading cuts. We define the rank of a node to be its number of good children (i.e., we do not count the bad children).

To meld two heaps, we link the corresponding trees, making the root that becomes a child bad. This melding does not change the rank of any node. To perform *find min*, we return the root of the tree representing the heap. To perform *delete min*, we delete the tree root, link pairs of trees whose roots have equal rank until no such linking is possible, making each node that becomes a nonroot good, and then link all the remaining trees in any order, making each node that becomes a nonroot bad. To perform *decrease key*, we perform the appropriate cuts (using mark bits as before), producing a set of trees. We link these trees in any order, making each root that becomes a nonroot bad. To perform an arbitrary deletion, we do the appropriate cuts, discard the node to be deleted, and combine the remaining trees using links as in *delete min*, first combining trees with roots of equal rank and marking the new nonroots good, then combining trees of unequal rank and marking the new nonroots bad.

The one-tree variant of F-heaps has the following properties: Corollary 1 still holds; that is, any node with k good children has at least $F_{k+2} \geq \phi^k$ descendants. The number of bad children created during the running of the algorithm is at most one per *insert* or *meld*, one per cut (and hence $O(1)$ per *decrease key*), and at most $O(\log n)$ per *delete min* or *delete*. Combining this with the previous analysis of F-heaps, we obtain an amortized time bound of $O(1)$ for *insert*, *meld*, *find min*, and *decrease key*, and $O(\log n)$ for *delete min* and *delete*.

Our fourth and last variant of F-heaps, called *F-heaps with implicit keys*, is designed to support the following additional heap operation:

increase all keys (Δ , h): Increase the keys of all items in heap h by the arbitrary real number Δ .

To implement *increase all keys*, we represent the keys of the items implicitly rather than explicitly, using a separate data structure. A suitable variant of compressed trees [23] suffices for this purpose. We maintain a compressed tree for each heap. Each node in the tree contains a value and possibly an item. Any node

FIG. 9. A compressed tree. Items are a , b , c , and d . The key of item a is $3 + 4 - 2 + 10 = 15$.

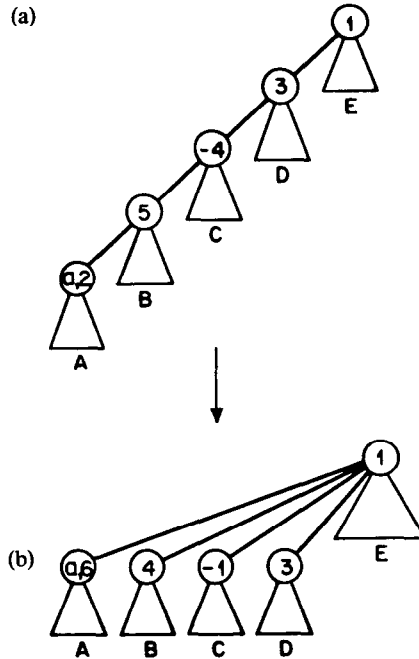
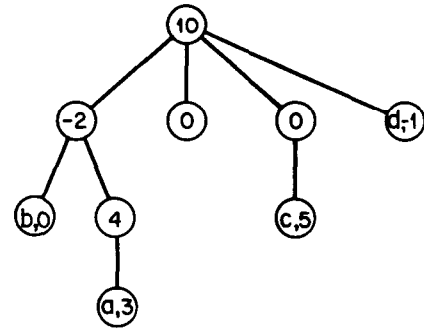


FIG. 10. Path compression. Triangles denote subtrees. (a) The original tree. (b) After evaluating key a .

containing an item has no children. The values represent the keys as follows: If x is any node containing an item i , the sum of the values of the ancestors of x (including x itself) is the key of i . (See Figure 9.)

We manipulate this data structure as follows: When executing *make heap*, we construct a new one-node compressed tree to represent the heap. The new node has value zero. When executing *insert* (i, h), we create a new compressed tree node x containing i , make the root of the compressed tree representing h the parent $p(x)$ of x , and give x a value defined by $value(x) = key(i) - value(p(x))$. When deleting an item i from a heap, we destroy the compressed tree node containing i . When executing *decrease key* (Δ, i, h), we subtract Δ from the value of the compressed tree node containing i . To perform *increase all keys* (Δ, h), we add Δ to the value of the root of the compressed tree representing h .

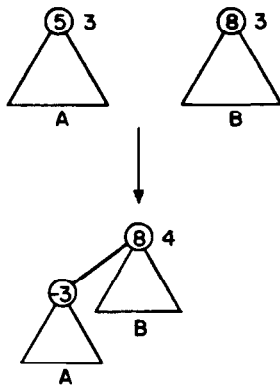


FIG. 11. Combining two compressed trees. Ranks are outside the roots.

Whenever we need to evaluate the key of an item i (as in a *find min* operation or a linking step), we locate the compressed tree node x containing i and follow the path from x through its ancestors up to the tree root. Then we walk back down the path from the root to x , *compressing* it as follows: When we visit a node y that is not a child of the root, we replace $value(y)$ by $value(y) + value(p(y))$ and redefine $p(y)$ to be the root. (See Figure 10.) This compression makes every node along the path a child of the root and preserves the relationship between values and keys. After the compression, we return $value(x) + value(p(x))$ as the key of i .

The last operation we must consider is melding. (If no melding takes place, then the depth of every compressed tree is at most one, and each operation on a compressed tree takes $O(1)$ time.) To facilitate melding we maintain for each compressed tree root a nonnegative integer *rank*. (This rank should not be confused with the rank of a heap-ordered tree node; it does not count the number of children.) A newly created compressed tree root has rank zero. When executing $meld(h_1, h_2)$, we locate the roots, say, x and y , of the compressed trees representing h_1 and h_2 . If $rank(x) > rank(y)$, we make x the parent of y and redefine $value(y)$ to be $value(y) - value(x)$. If $rank(x) < rank(y)$, we make y the parent of x and redefine $value(x)$ to be $value(x) - value(y)$. Finally, if $rank(x) = rank(y)$, we increase $rank(x)$ by one and proceed as in the case of $rank(x) > rank(y)$. (See Figure 11.)

To implement this data structure, we need one compressed tree node for each *make heap* operation plus one node per item, with room in each node for an item or rank, a value, and a parent pointer. The total time for compressed tree operations is $O(m + f\alpha(m + f, n))$, where m is the number of heap operations, n is the number of *make heap* and *insert* operations, f is the number of key evaluations, and α is a functional inverse of Ackerman's function [23, 26]. In most applications of heaps requiring use of the *increase all keys* operation, the time for manipulating heap-ordered trees will dominate the time for manipulating compressed trees. Note that the two data structures are entirely separate; we can use compressed trees in combination with any implementation of heaps that is based on key comparison.

4. Shortest Paths

In this section we use F-heaps to implement Dijkstra's shortest path algorithm [5] and explore some of the consequences. Our discussion of Dijkstra's algorithm is based on Tarjan's presentation [24]. Let G be a directed graph, one of whose vertices is distinguished as the *source* s , and each of whose edges (v, w) has a

nonnegative length $l(v, w)$. The *length* of a path in G is the sum of its edge lengths; the *distance* from a vertex v to a vertex w is the minimum length of a path from v to w . A minimum-length path from v to w is called a *shortest path*. The *single-source shortest path problem* is that of finding a shortest path from s to v for every vertex v in G .

We shall denote the number of vertices in G by n and the number of edges by m . We assume that there is a path from s to any other vertex; thus $m \geq n - 1$. Dijkstra's algorithm solves the shortest path problem using a *tentative distance function* d from vertices to real numbers with the following properties:

- (1) For any vertex v such that $d(v)$ is finite, there is a path from s to v of length $d(v)$;
- (2) when the algorithm terminates, $d(v)$ is the distance from s to v .

Initially $d(s) = 0$ and $d(v) = \infty$ for $v \neq s$. During the running of the algorithm, each vertex is in one of three states: *unlabeled*, *labeled*, or *scanned*. Initially s is labeled and all other vertices are unlabeled. The algorithm consists of repeating the following step until there are no labeled vertices (every vertex is scanned):

Scan. Select a labeled vertex v with $d(v)$ minimum. Convert v from labeled to scanned. For each edge (v, w) such that $d(v) + l(v, w) < d(w)$, replace $d(w)$ by $d(v) + l(v, w)$ and make w labeled.

The nonnegativity of the edge lengths implies that a vertex, once scanned, can never become labeled, and further that the algorithm computes distances correctly.

To implement Dijkstra's algorithm, we use a heap to store the set of labeled vertices. The tentative distance of a vertex is its key. Initialization requires one *make heap* and one *insert* operation. Each scanning step requires one *delete min* operation. In addition, for each edge (v, w) such that $d(v) + l(v, w) < d(w)$, we need either an *insert* operation (if $d(w) = \infty$) or a *decrease key* operation (if $d(w) < \infty$). Thus there is one *make heap* operation, n *insert* and n *delete min* operations, and at most m *decrease key* operations. The maximum heap size is $n - 1$. If we use an F-heap, the total time for heap operations is $O(n \log n + m)$. The time for other tasks is $O(n + m)$. Thus we obtain an $O(n \log n + m)$ running time for Dijkstra's algorithm. This improves Johnson's bound of $O(m \log_{(m/n+2)} n)$ [13, 24] based on the use of implicit heaps with a branching factor of $m/n + 2$.

By augmenting the algorithm, we can make it compute shortest paths as well as distances: For each vertex v , we maintain a *tentative predecessor* $pred(v)$ that immediately precedes v on a tentative shortest path from s to v . When replacing $d(w)$ by $d(v) + l(v, w)$ in a scanning step, we define $pred(w) = v$. Once the algorithm terminates, we can find a shortest path from s to v for any vertex v by following predecessor pointers from v back to s . Augmenting the algorithm to maintain predecessors adds only $O(m)$ to the running time.

Several other optimization algorithms use Dijkstra's algorithm as a subroutine, and for each of these, we obtain a corresponding improvement. We shall give three examples:

- (1) The all-pairs shortest path problem, with or without negative edge lengths, can be solved in $O(nm)$ time plus n iterations of Dijkstra's algorithm [13, 24]. Thus we obtain a time bound of $O(n^2 \log n + nm)$, improved from $O(nm \log_{(m/n+2)} n)$.
- (2) The assignment problem (bipartite weighted matching) can also be solved in $O(nm)$ time plus n iterations of Dijkstra's algorithm [24]. Thus we obtain a bound of $O(n^2 \log n + nm)$, improved from $O(nm \log_{(m/n+2)} n)$.

- (3) Knuth's generalization [16] of Dijkstra's algorithm to compute minimum-cost derivations in a context-free language runs in $O(n \log n + t)$ time, improved from $O(m \log n + t)$, where n is the number of nonterminals, m is the number of productions, and t is the total length of the productions. (There is at least one production per nonterminal; thus $m \geq n$.)

5. Minimum Spanning Trees

A less immediate application of F-heaps is to the computation of minimum spanning trees. See [24] for a systematic discussion of minimum spanning tree algorithms. Let G be a connected undirected graph with n vertices and m edges (v, w) each of which has a nonnegative cost $c(v, w)$. A *minimum spanning tree* of G is a spanning tree of G of minimum total edge cost.

We can find a minimum spanning tree by using a generalized greedy approach. We maintain a forest defined by the edges so far selected to be in the minimum spanning tree. We initialize the forest to contain each of the n vertices of G as a one-vertex tree. Then we repeat the following step $n - 1$ times (until there is only one n -vertex tree):

Connect. Select any tree T in the forest. Find a minimum-cost edge with exactly one endpoint in T and add it to the forest. (This connects two trees to form one.)

This algorithm is nondeterministic: We are free to select the tree to be processed in each connecting step in any way we wish. One possibility is to always select the tree containing a fixed start vertex s , thereby growing a single tree T that absorbs the vertices of G one by one. This algorithm, usually credited to Prim [18] and Dijkstra [5] independently, was actually discovered by Jarník [12] (see Graham and Hell's survey paper [11]). The algorithm is almost identical to Dijkstra's shortest path algorithm, and we can implement it in the same way. For each vertex v not yet in T , we maintain a key measuring the tentative cost of connecting v to T . If $v \neq s$ and $\text{key}(v) < \infty$, we also maintain an edge $e(v)$ by which the connection can be made. We start by defining $\text{key}(s) = 0$ and $\text{key}(v) = \infty$ for $v \neq s$. Then we repeat the following step until no vertex has finite key:

Connect to start. Select a vertex v with $\text{key}(v)$ minimum among vertices with finite key. Replace $\text{key}(v)$ by $-\infty$. For each edge (v, w) such that $c(v, w) < \text{key}(w)$, replace $\text{key}(w)$ by $c(v, w)$ and define $e(w) = (v, w)$.

When this algorithm terminates, the set of edges $e(v)$ with $v \neq s$ defines a minimum spanning tree. The purpose of setting $\text{key}(v) = -\infty$ in the connecting step is to mark v as being in T . If we store the vertices with finite key in an F-heap, the algorithm requires n *delete min* operations and $O(m)$ other heap operations, none of them deletions. The total running time is $O(n \log n + m)$.

The best previous bound for computing minimum spanning trees is $O(m \log \log_{(m/n+2)} n)$ [4, 24], a slight improvement over Yao's $O(m \log \log n)$ bound [28]. Our $O(n \log n + m)$ bound is better than the old bound for graphs of intermediate density (e.g., $m = \Theta(n \log n)$). However, we can do better.

The idea is to grow a single tree only until its heap of neighboring vertices exceeds a certain critical size. Then we start from a new vertex and grow another tree, again stopping when the heap gets too large. We continue in this way until every vertex is in a tree. Then we condense every tree into a single supervertex and begin a new pass of the same kind over the condensed graph. After a sufficient number of passes, only one supervertex will remain, and by expanding the supervertices, we can extract a minimum spanning tree.

Our implementation of this method does the condensing implicitly. We shall describe a single pass of the algorithm. We begin with a forest of previously grown trees, which we call *old trees*, defined by the edges so far added to the forest. The pass connects these old trees into new larger trees that become the old trees for the next pass.

To start the pass, we number the old trees consecutively from one and assign to each vertex the number of the tree containing it. Thus allows us to refer to trees by number and to directly access the old tree $tree(v)$ containing any vertex v . Next, we do a *cleanup*, which takes the place of condensing. We discard every edge connecting two vertices in the same old tree and all but a minimum-cost edge connecting each pair of old trees. We can do such a cleanup in $O(m)$ time by sorting the edges lexicographically on the numbers of the trees containing their endpoints, using a two-pass radix sort. Then we scan the sorted edge list, saving only the appropriate edges. After the cleanup we construct a list for each old tree T of the edges with one endpoint in T . (Each edge will appear in two such lists.)

To grow new trees, we give every old tree a key of ∞ and unmark it. We create an empty heap. Then we repeat the following tree-growing step until there are no unmarked old trees. This completes the pass.

Grow a New Tree. Select any unmarked old tree T_0 , and insert it as an item into the heap with a key of zero. Repeat the following step until the heap is empty, or it contains more than k trees (where k is a parameter to be chosen later), or the growing tree becomes connected to a marked old tree:

Connect to Starting Tree. Delete an old T of minimum key from the heap. Set $key(T) = -\infty$. If $T \neq T_0$ (i.e., T is not the starting tree of this growth step), add $e(T)$ to the forest. (Edge $e(T)$ connects old tree T to the current tree containing T_0 .) If T is marked, stop growing the current tree, and finish the growth step as described below. Otherwise, mark T . For each edge (v, w) with v in T and $c(v, w) < key(tree(w))$, set $e(tree(w)) = (v, w)$. If $key(tree(w)) = \infty$, insert $tree(w)$ in the heap with a redefined key of $c(v, w)$; if $c(v, w) < key(tree(w)) < \infty$, decrease the key of $tree(w)$ to $c(v, w)$.

To finish the growth step, empty the heap and set $key(T) = \infty$ for every old tree T with finite key (these are the trees that have been inserted into the heap during the current growth step).

We can analyze the running time of a pass as follows: The time for the cleanup and other initialization is $O(m)$. If t is the number of old trees, the total time for growing new trees is $O(t \log k + m)$: We need at most t *delete min* operations, each on a heap of size k or smaller, and $O(m)$ other heap operations, none of which is a deletion.

We wish to choose values of k for successive passes so as to minimize the total running time. Smaller values of k reduce the time per pass; larger values reduce the number of passes. For each pass let us choose $k = 2^{2m/t}$, where m is the original number of edges in the graph and t is the number of trees before the pass. The value of k increases from pass to pass as the number of trees decreases. With this choice of k , the running time per pass is $O(m)$.

It remains for us to bound the number of passes. Consider the effect of a pass that begins with t trees and $m' \leq m$ edges (some edges may have been discarded). Each tree T remaining after the pass has more than $k = 2^{2m'/t}$ edges with at least one endpoint in T . (If T_0 is the first old tree among those making up T that was placed in the heap, then T_0 was grown until the heap reached size k , at which time

the current tree T' containing T_0 had more than k incident edges. Other trees may have later been connected to T' , causing some of these incident edges to have *both* their endpoints in the final tree T .) Since each of the m' edges has only two endpoints, this means that the number of trees remaining after the pass, say, t' , satisfies $t' \leq 2m'/k$. If k' is the heap size bound for the next pass, we have $k' = 2^{2m/t'} \geq 2^k$. Since the initial heap size bound is $2m/n$ and a heap size bound of n or more is only possible on the last pass, the number of passes is at most $\min\{i \mid \log^{(i)} n \leq 2m/n\} + 1 = \beta(m, n) + O(1)$, where $\beta(m, n) = \min\{i \mid \log^{(i)} n \leq m/n\}$ and $\log^{(i)} n$ is defined inductively by $\log^{(0)} n = n$, $\log^{(i+1)} n = \log \log^{(i)} n$.

Thus we obtain a time bound of $O(m\beta(m, n))$ for the computation of minimum spanning trees. This bound improves the old $O(m \log \log_{(m/n+2)} n)$ bound for all sufficiently sparse graphs. Note that $\beta(m, n) \leq \log^* n$ if $m \geq n$, where $\log^* n = \min\{i \mid \log^{(i)} n \leq 1\}$. (If $m < n$, then $m = n - 1$ and the entire graph is a tree, since we have assumed that the graph is connected.)

Our fast minimum spanning tree algorithm improves the time bounds for certain kinds of constrained minimum spanning tree problems as well. For example, one can find a minimum spanning tree with a degree constraint at one vertex in $O(m\beta(m, n))$ time, and a minimum spanning tree with a fixed number of marked edges in a graph with marked and unmarked edges in $O(n \log n + m)$ time. For details, See Gabow and Tarjan's paper [8].

6. Recent Results and Open Problems

F-heaps have several additional applications. H. Gabow (private communication, Oct. 1984) has noted that they can be used to speed up the scaling algorithm of Edmonds and Karp [6] for minimum-cost network flow from $O(m^2(\log_{(m/n+2)} n)(\log N))$ to $O(m(n \log n + m)(\log N))$, where N is the maximum capacity, assuming integer capacities. They can also be used to find shortest pairs of disjoint paths in $O(n \log n + m)$ time [7], improved from $O(m \log_{(m/n+2)} n)$ [20].

A less immediate application is to the directed analogue of the minimum spanning tree problem—the optimum branching problem. Gabow, Galil, and Spencer [9] have proposed a very complicated $O(n \log n + m \log \log \log_{(m/n+2)} n)$ -time algorithm for this problem, improving on Tarjan's [3, 21] bound of $O(\min\{m \log n, n^2\})$. F-heaps can be used to solve this problem in $O(n \log n + m)$ time [10].

Another recent major result is an improvement by Gabow, Galil, and Spencer [9] to our minimum spanning tree algorithm of Section 4. They have improved our time bound from $O(m\beta(m, n))$ to $O(m \log \beta(m, n))$ by introducing the idea of grouping edges with a common vertex into “packets” and working only with packet minima. (See [10].)

Several intriguing open questions are raised by our work:

(i) Is there a “self-adjusting” version of F-heaps that achieves the same amortized time bounds, but requires neither maintaining ranks nor performing cascading cuts? The self-adjusting version of leftist heaps proposed by Sleator and Tarjan [19] does not solve this problem, as the amortized time bound for *decrease key* is $O(\log n)$ rather than $O(1)$. We have a candidate data structure, but are so far unable to verify that it has the desired time bounds.

(ii) Can the best time bounds for finding shortest paths and minimum spanning trees be improved? Dijkstra's algorithm requires $\Omega(n \log n + m)$ time assuming a comparison-based computation model, since it must examine all the edges in the worst case and can be used to sort n numbers. Nevertheless, this does not preclude

the existence of another, faster algorithm. Similarly, there is no reason to suppose that the Gabow–Galil–Spencer bound for minimum spanning trees is the best possible. It is suggestive that the minimality of a spanning tree can be checked in $O(m\alpha(m, n))$ time [23] and even in $O(m)$ comparisons [17]. Furthermore, if the edges are presorted, a minimum spanning tree can be computed in $O(m\alpha(m, n))$ time [24].

(iii) Are there other problems needing heaps where the use of F-heaps gives asymptotically improved algorithms? A possible candidate is the nonbipartite weighted matching problem, for which the current best bound of $O(n^2 \log n + nm \log \log \log_{(m/n+2)} n)$ [9] might be improvable to $O(n^2 \log n + nm)$.

ACKNOWLEDGMENT. We thank Dan Sleator for perceptive suggestions that helped to simplify our data structure and Hal Gabow for pointing out the additional applications of F-heaps mentioned in Section 6.

REFERENCES

1. AHO, A. V., HOPCROFT, J. E., AND ULLMAN, J. D. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, Reading, Mass. 1974.
2. BROWN, M. R. Implementation and analysis of binomial queue algorithms. *SIAM J. Comput.* 7, 3 (Aug. 1978), 298–319.
3. CAMERINI, P. M., FRATTA, L., AND MAFFIOLI, F. A note on finding optimum branchings. *Networks* 9, 4 (Winter 1979), 309–312.
4. CHERITON, D., AND TARJAN, R. E. Finding minimum spanning trees. *SIAM J. Comput.* 5, 4 (Dec. 1976), 724–742.
5. DIJKSTRA, E. W. A note on two problems in connexion with graphs. *Numer. Math.* 1, 4 (Sept. 1959), 269–271.
6. EDMONDS, J., AND KARP, R. M. Theoretical improvements in algorithmic efficiency for network flow problems. *J. ACM* 19, 2 (Apr. 1972), 248–264.
7. GABOW, H. N., AND STALLMAN, M. Efficient algorithms for the parity and intersection problems on graphic matroids. In *Automata, Languages, and Programming, 12th Colloquium, Lecture Notes in Computer Science*, Vol. 194, W. Brauer, Ed. Springer-Verlag, New York, 1985, pp. 210–220.
8. GABOW, H. N., AND TARJAN, R. E. Efficient algorithms for a family of matroid intersection problems. *J. Algorithms* 5, 1 (Mar. 1984), 80–131.
9. GABOW, H. N., GALIL, Z., AND SPENCER, T. Efficient implementation of graph algorithms using contraction. In *Proceedings of the 25th Annual IEEE Symposium on Foundations of Computing*. (Singer Island, Fla., Oct. 24–26). IEEE Press, New York, 1984 pp. 338–346.
10. GABOW, H. N., GALIL, Z., SPENCER, T., AND TARJAN, R. E. Efficient algorithms for finding minimum spanning trees in undirected and directed graphs. *Combinatorica* 6, 2 (1986), 109–122.
11. GRAHAM, R. L., AND HELL, P. On the history of the minimum spanning tree problem. *Ann. Hist. Comput.* 7, 1 (Jan. 1985), 43–57.
12. JARNÍK, V. O jistém problému minimálním. *Práce Moravské Přírodovědecké Společnosti* 6 (1930), 57–63 (in Czechoslovakian).
13. JOHNSON, D. B. Efficient algorithms for shortest paths in sparse networks. *J. ACM* 24, 1 (Jan. 1977), 1–13.
14. KNUTH, D. E. *The Art of Computer Programming*. Vol. 1, *Fundamental Algorithms*, 2nd ed. Addison-Wesley, Reading, Mass. 1973.
15. KNUTH, D. E. *The Art of Computer Programming*. Vol. 3, *Sorting and Searching*. Addison-Wesley, Reading, Mass., 1973.
16. KNUTH, D. E. A generalization of Dijkstra's algorithm. *Inf. Process. Lett.* 6, 1 (Feb. 1977), 1–5.
17. KOMLÓS, J. Linear verification for spanning trees. In *Proceedings of the 25th Annual IEEE Symposium on Foundations of Computing*. (Singer Island, Fla., Oct. 24–26). IEEE Press, New York, 1984 pp. 201–206.
18. PRIM, R. C. Shortest connection networks and some generalizations. *Bell Syst. Tech. J.* 36, 6 (Nov. 1957), 1389–1401.
19. SLEATOR, D. D., AND TARJAN, R. E. Self-adjusting heaps. *SIAM J. Comput.* 15, 1 (Feb. 1986), 52–69.

20. SUURBALLE, J. W., AND TARJAN, R. E. A quick method for finding shortest pairs of paths. *Networks* 14 (1984), 325-336.
21. TARJAN, R. E. Finding optimum branchings. *Networks* 7, 1 (Spring 1977), 25-35.
22. TARJAN, R. E. A class of algorithms which require nonlinear time to maintain disjoint sets. *J. Comput. Syst. Sci.* 18, 2 (Apr. 1979), 110-127.
23. TARJAN, R. E. Applications of path compression on balanced trees. *J. ACM* 26, 4 (Oct. 1979), 690-715.
24. TARJAN, R. E. *Data Structures and Network Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, Pa., 1983.
25. TARJAN, R. E. Amortized computational complexity. *SIAM J. Algebraic Discrete Methods* 6, 2 (Apr. 1985), 306-318.
26. TARJAN, R. E., AND VAN LEEUWEN, J. Worst-case analysis of set union algorithms. *J. ACM* 31, 2 (1984), 245-281.
27. VUILLEMIN, J. A data structure for manipulating priority queues. *Commun. ACM* 21, 4 (Apr. 1978), 309-315.
28. YAO, A. An $O(|E| \log \log |V|)$ algorithm for finding minimum spanning trees. *Inf. Process. Lett.* 4, 1 (Sept. 1975), 21-23.

RECEIVED NOVEMBER 1984; REVISED FEBRUARY 1986; ACCEPTED JUNE 1986